
Uninformed Search Example 1: BFS in Grid

Find the shortest path from top-left to bottom-right in a grid using BFS.

Python Code:

```
from collections import deque

def bfs_grid(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    visited = set()
    queue = deque([(start, [start])])

    while queue:
        (x, y), path = queue.popleft()
        if (x, y) == goal:
            return path
        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nx, ny = x+dx, y+dy
            if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 0 and
            (nx, ny) not in visited:
                visited.add((nx, ny))
                queue.append(((nx, ny), path + [(nx, ny)]))
    return None
```

Uninformed Search Example 2: DFS in Maze

Use DFS to search through a maze represented as a grid.

Python Code:

```

def dfs_maze(grid, x, y, path, visited, goal):
    if (x, y) == goal:
        return path
    visited.add((x, y))
    for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
        nx, ny = x+dx, y+dy
        if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]) and grid[nx][ny]
        == 0 and (nx, ny) not in visited:
            result = dfs_maze(grid, nx, ny, path + [(nx, ny)], visited, goal)
            if result:
                return result
    return None

```

Uninformed Search Example 3: UCS on Weighted Graph

Find the lowest cost path using UCS on a weighted graph.

Python Code:

```

import heapq

def ucs(graph, start, goal):
    pq = [(0, [start])]
    visited = set()
    while pq:
        cost, path = heapq.heappop(pq)
        node = path[-1]
        if node == goal:
            return path, cost
        if node not in visited:
            visited.add(node)
            for neighbor, weight in graph[node]:
                heapq.heappush(pq, (cost + weight, path + [neighbor]))
    return None

```

Uninformed Search Example 4: BFS Tree Traversal

Traverse a binary tree level-by-level using BFS.

Python Code:

```
def bfs_tree(root):  
    from collections import deque  
    if not root:  
        return []  
    queue = deque([root])  
    result = []  
    while queue:  
        node = queue.popleft()  
        result.append(node.val)  
        if node.left:  
            queue.append(node.left)  
        if node.right:  
            queue.append(node.right)  
    return result
```

Uninformed Search Example 5: DFS Tree Traversal

Use DFS to traverse a binary tree (Preorder).

Python Code:

```
def dfs_preorder(root):  
    if not root:  
        return []  
    return [root.val] + dfs_preorder(root.left) + dfs_preorder(root.right)
```

Informed Search Example 1: Greedy BFS in Grid

Find a goal using Greedy BFS in a grid with Manhattan heuristic.

Python Code:

```
import heapq

def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def greedy_bfs(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    visited = set()
    pq = [(heuristic(start, goal), [start])]
    while pq:
        _, path = heapq.heappop(pq)
        x, y = path[-1]
        if (x, y) == goal:
            return path
        if (x, y) not in visited:
            visited.add((x, y))
            for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                nx, ny = x+dx, y+dy
                if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 0:
                    heapq.heappush(pq, (heuristic((nx, ny), goal), path + [(nx,
ny)]))
    return None
```

Informed Search Example 2: A* Search in Grid

Find the optimal path in a grid using A* search.

Python Code:

```
def astar_grid(grid, start, goal):
    import heapq
    def heuristic(a, b):
        return abs(a[0] - b[0]) + abs(a[1] - b[1])
    rows, cols = len(grid), len(grid[0])
    pq = [(heuristic(start, goal), 0, [start])]
    visited = set()
```

```

while pq:
    est, g, path = heapq.heappop(pq)
    x, y = path[-1]
    if (x, y) == goal:
        return path, g
    if (x, y) not in visited:
        visited.add((x, y))
        for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
            nx, ny = x+dx, y+dy
            if 0 <= nx < rows and 0 <= ny < cols:
                new_g = g + grid[nx][ny]
                heapq.heappush(pq, (new_g + heuristic((nx, ny), goal), new_g,
path + [(nx, ny)]))
    return None

```

Informed Search Example 3: A* on Graph

Apply A* on a weighted graph using heuristic values.

Python Code:

```

def astar_graph(graph, heuristic, start, goal):
    import heapq
    pq = [(heuristic[start], 0, [start])]
    visited = set()
    while pq:
        est, g, path = heapq.heappop(pq)
        node = path[-1]
        if node == goal:
            return path, g
        if node not in visited:
            visited.add(node)
            for neighbor, cost in graph[node]:
                new_g = g + cost
                heapq.heappush(pq, (new_g + heuristic[neighbor], new_g, path
+ [neighbor]))
    return None

```

Informed Search Example 4: Greedy Word Ladder

Use GBFS to transform one word into another (1-letter difference).

Python Code:

```
def greedy_word_ladder(words, start, goal):
    import heapq
    def h(w): return sum(1 for a, b in zip(w, goal) if a != b)
    pq = [(h(start), [start])]
    visited = set()
    while pq:
        _, path = heapq.heappop(pq)
        current = path[-1]
        if current == goal:
            return path
        visited.add(current)
        for neighbor in words.get(current, []):
            if neighbor not in visited:
                heapq.heappush(pq, (h(neighbor), path + [neighbor]))
    return None
```

Informed Search Example 5: A* Search in Weighted Maze

Navigate a weighted maze using A* to minimize total cost.

Python Code:

```
# Similar to astar_grid but grid cells have weights and heuristic uses
# Manhattan distance.
# Use astar_grid from Example 2 above with a grid like:
# grid = [[1, 3, 1], [2, 1, 5], [3, 1, 1]]
```
